Unidad 4 - Actividad Práctica - PARTE 2

Metodología de Sistemas II - TUP 2026 - Proyecto “Educar para Transformar”

Equipo: Trangoni, Fabián - Mazza, Bruno

Repositorio: https://github.com/Brunomazza/metodologia-2

Entrega para coevaluación: 27 de octubre

1. Funcionalidad seleccionada

Gestión de Usuarios del sistema (administración), conectada a Firebase (pantallas *admin-usuarios*, *setup-usuarios* y *login-admin*; Firebase Authentication y Firestore). Corresponde al último avance del repositorio (commit 065f186). El proyecto es un sitio HTML/CSS/JS con Firebase y un backend Express (*API_Server.js*).

2. Análisis de los contenidos de la Unidad 4

Estado según lo que realmente existe hoy en el repositorio (rama main, 22/09/2026):

Contenido	Estado	Herramientas / estrategias	Flujo
Integración Continua	No aplicado	No existe carpeta .github ni workflows. Se aplicará (punto 3).	-
Despliegue Continuo	No aplicado	Todavía no hay despliegue automatizado.	-
Build	No aplicado	Sitio estático: no requiere compilación. El backend se ejecuta con *npm start*.	-
Pipelines	No aplicado	Se aplicará junto con la CI (punto 3).	-
Variables de Entorno	Parcial	*.env* ignorado en Backend/.gitignore; el puerto se lee de *process.env.PORT*. La configuración de Firebase está en js/firebase-config.js.	Se define PORT -> API_Server.js escucha en ese puerto.
Gestión de Artefactos	Parcial	Dependencias con npm y package-lock.json; node_modules fuera del repositorio. No se generan artefactos de build.	npm ci -> instala las versiones exactas del lock.
Integración repo - CI/CD	No aplicado	Repositorio en GitHub sin Actions conectado. Se aplicará.	-
Pull Requests / Merge Requests	No aplicado	Hasta hoy: 0 Pull Requests; todos los cambios van directo a main. Se aplicará (punto 3).	-
Issues	Parcial	Las historias de usuario (HU-01 a HU-06) están definidas en el backlog del Plan de Trabajo; no se usan GitHub Issues.	Historia de usuario -> tarjeta en el tablero -> desarrollo.
Wikis	Parcial	README.md y carpeta *Entregas* con la documentación en PDF.	Se completa una unidad -> se agrega el PDF a Entregas -> commit.
Tablero Kanban	Aplicado	Jira con tablero Kanban por equipo y por integrante, definido en el Plan de Trabajo (columnas To Do, In Progress, Done).	Tarjeta HU-xx: To Do -> In Progress -> Done al fusionar el cambio.
Otros	Aplicado	Git/GitHub, .gitignore, commits descriptivos, API REST con Express.	Cambio local -> commit -> push a main.

3. Contenidos no aplicados que se aplicarán

Se eligieron **Pull Requests (con aprobación de otro integrante)** e **Integración Continua con GitHub Actions**.

Justificación. El historial muestra 8 commits directos a main, sin revisión y sin validación automática, concentrados en pocos días, con commits de corrección posteriores (“Pequeños errores”, “Otros pequeños cambios”). Los Pull Requests aportan revisión entre pares y la CI verifica cada cambio antes de integrarlo; ambas prácticas se complementan y son de bajo costo para un equipo de dos personas.

Aplicación:

- Workflow *.github/workflows/ci.yml* (se adjunta): en cada Pull Request a main instala dependencias (*npm ci*), verifica la sintaxis del backend y de los scripts del frontend, inicia la API y comprueba que */api/alumnos* y */api/dashboard* respondan correctamente. Fue probado localmente.
- Regla de protección de la rama main: exigir Pull Request, 1 aprobación de otro integrante y CI en verde para poder fusionar.
- Trabajo en ramas *feature/HU-xx* (con el código de la historia de usuario), un Pull Request por cambio, vinculado a su tarjeta del tablero Kanban.
- Revisión semanal del tablero (15 minutos) para mantener el estado de las tarjetas al día.

Flujo: tarjeta del tablero (To Do -> In Progress) -> rama feature/HU-xx -> push -> Pull Request a main -> GitHub Actions ejecuta la CI -> el otro integrante revisa y aprueba -> merge a main -> tarjeta a Done.

Estado: aplicación en curso. Las capturas de los Pull Requests y de las ejecuciones de Actions se presentan el 27/10.

4. Métricas de productividad

Herramientas: historial de Git (*git log*) y las pestañas Pull requests y Actions de GitHub.

Métricas: Lead Time (desde el primer commit de una rama hasta el merge), Pull Requests integrados por semana y porcentaje de cambios aprobados por otra persona. En lugar de Deployment Frequency se usa la integración semanal de Pull Requests, porque todavía no hay despliegue.

Resultados iniciales (datos reales, git log del 01/09 al 22/09/2026):

- 8 commits: Fabián Trangoni 5, Bruno Mazza 2 y 1 commit inicial.
- 0 Pull Requests y una sola rama: el 100% de los commits va directo a main. El Lead Time por Pull Request no es medible.
- Dos commits el 01/09, luego casi 20 días sin integrar cambios (hasta el 21/09).
- 6 de los 8 commits (75%) se hicieron en unas 25 horas, entre el 21/09 y el 22/09.

Análisis: el equipo integra tarde y todo junto, sin revisión previa ni validación automática. Esto aumenta el riesgo de conflictos y errores y genera commits de corrección posteriores.

Estrategias de mejora: (1) Pull Requests pequeños con aprobación obligatoria de otro integrante; (2) CI automática que valide cada cambio antes del merge; (3) integrar como mínimo dos veces por semana.

Tabla comparativa (la columna “Objetivo” es la meta; “Después” se completa con datos reales el 27/10):

Métrica	Antes (real)	Objetivo	Después
Pull Requests integrados por semana	0	2 o más	A completar el 27/10
Cambios aprobados por otro integrante	0 (sin PRs)	100%	A completar el 27/10
Cambios validados por CI antes del merge	0%	100%	A completar el 27/10
Commits directos a main	100% (8 de 8)	0%	A completar el 27/10
Lead Time por Pull Request	No medible	Menos de 24 h	A completar el 27/10
Máximo de días sin integrar	Casi 20	7 o menos	A completar el 27/10

5. Plan de trabajo para aplicar las mejoras

Fecha	Responsable	Acción
Mié 07/10	Mazza	Crear la rama ci/github-actions, agregar ci.yml y abrir el Pull Request #1.
Jue 08/10	Trangoni	Revisar y aprobar el PR #1, fusionar a main y activar la regla de protección de la rama.
Vie 09/10 - Sáb 10/10	Ambos	Un Pull Request de mejora cada uno (ramas feature/HU-xx), con su tarjeta en el tablero, aprobado por el otro con la CI en verde.
Dom 11/10 y semanas siguientes	Ambos	Seguir integrando por Pull Requests, actualizar el tablero Kanban en la revisión semanal y registrar las métricas con *gh pr list*.

Fecha	Responsable	Acción
Lun 26/10	Ambos	Completar la columna “Después” y las capturas de evidencia para la coevaluación del 27/10.

Anexo - Comandos para obtener las métricas

```
git log --pretty=format:"%h | %an | %ad | %s" --date=iso
gh pr list --state merged --json number,title,author,createdAt,mergedAt
gh run list --limit 30
```